AI ASSISTED CODING

ASSIGNMENT-17.3

Name: V.Sruthi

Roll No: 2303A51308

Batch:05

Task 1 Employee Dataset Cleaning

Task: Use AI assistance to clean an employee information dataset.

Prompt:

Clean an employee dataset using Python (Pandas &Scikit-learn): handle missing values (salary → median, designation → mode, experience → median), remove duplicate employee_id, convert text columns to lowercase, and encode designation (label encoding) and employment_type (one-hot encoding). Generate clean, complete code.

T **Prompt:** Clean an employee dataset using Python (Pandas & Scikit-learn): handle missing values (salary → median, designation → mode, experience → median), remove duplicate employee_id, convert text columns to lowercase, and encode designation (label encoding) and employment_type (one-hot encoding). Generate clean, complete code.

Dataset:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# Create a sample DataFrame for demonstration purposes
data = {
    'employee_id': [101, 102, 103, 104, 105, 101, 106, 107, 108, 109],
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Alice', 'Frank', 'Grace', 'Heidi', 'Ivan'],
    'department': ['HR', 'IT', 'Finance', 'IT', 'Marketing', 'HR', 'IT', 'Finance', 'HR', 'Marketing'],
    'designation': ['Manager', 'Developer', 'Analyst', 'Developer', 'Marketing Specialist', 'Manager', 'Developer', np.nan, 'HR Assist
    'salary': [70000, 85000, 60000, np.nan, 75000, 70000, 90000, 62000, np.nan, 68000],
    'experience': [5, 8, 3, 7, np.nan, 5, 9, 4, 2, 6],
    'employment_type': ['Full-Time', 'Contract', 'Full-Time', 'Part-Time', 'Full-Time', 'Full-Time', 'Contract', 'Full-Time', 'Part-Ti
}
df = pd.DataFrame(data)

print("Original DataFrame:")
display(df.head(10))
print("\nOriginal DataFrame Info:")
df.info()
```

```
Original DataFrame:
```

	employee_id	name	department	designation	salary	experience	employment_type
0	101	Alice	HR	Manager	70000.0	5.0	Full-Time
1	102	Bob	IT	Developer	85000.0	8.0	Contract
2	103	Charlie	Finance	Analyst	60000.0	3.0	Full-Time
3	104	David	IT	Developer	NaN	7.0	Part-Time
4	105	Eve	Marketing	Marketing Specialist	75000.0	NaN	Full-Time
5	101	Alice	HR	Manager	70000.0	5.0	Full-Time
6	106	Frank	IT	Developer	90000.0	9.0	Contract
7	107	Grace	Finance	NaN	62000.0	4.0	Full-Time
8	108	Heidi	HR	HR Assistant	NaN	2.0	Part-Time

Code:

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
# 2. Handle Missing Values
df['salary'].fillna(df['salary'].median(), inplace=True)
df['designation'].fillna(df['designation'].mode()[0], inplace=True)
df['experience'].fillna(df['experience'].median(), inplace=True)
print("\nDataFrame Info after handling missing values:")
df.info()

# 3. Remove Duplicate Employee IDs
df.drop_duplicates(subset='employee_id', inplace=True)
print("\nDataFrame Info after removing duplicate 'employee_id':")
df.info()

# 4. Convert Text Columns to Lowercase
text_columns = ['name', 'department', 'designation', 'employment_type']
for col in text_columns:
    if col in df.columns and df[col].dtype == 'object':
        df[col] = df[col].str.lower()

# 5. Encode Categorical Features
# Label Encoding for 'designation'
le = LabelEncoder()
df['designation_encoded'] = le.fit_transform(df['designation'])

# One-Hot Encoding for 'employment_type'
df['employment_type'].fillna('unknown', inplace=True)
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
employment_type_encoded = ohe.fit_transform(df[['employment_type']])
employment_type_df = pd.DataFrame(employment_type_encoded, columns=ohe.get_feature_names_out(['employment_type']), index=df.index)
df = pd.concat([df, employment_type_df], axis=1)

# Drop original categorical columns
df.drop(columns=['designation', 'employment_type'], inplace=True)
```

Output:

```
df['employment_type'].fillna('unknown', inplace=True) # or choose another strategy
```

	employee_id	name	department	salary	experience	designation_encoded	employment_type_contract	employment_type_full-time	employment
0	101	alice	hr	70000.0	5.0	3	0.0	1.0	
1	102	bob	it	85000.0	8.0	1	1.0	0.0	
2	103	charlie	finance	60000.0	3.0	0	0.0	1.0	
3	104	david	it	70000.0	7.0	1	0.0	0.0	
4	105	eve	marketing	75000.0	5.0	4	0.0	1.0	
6	106	frank	it	90000.0	9.0	1	1.0	0.0	
7	107	grace	finance	62000.0	4.0	1	0.0	1.0	
8	108	heidi	hr	70000.0	2.0	2	0.0	0.0	
9	109	ivan	marketing	68000.0	6.0	0	0.0	0.0	

```
Cleaned DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 9 entries, 0 to 9
Data columns (total 10 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   employee_id                9 non-null      int64
 1   name                       9 non-null      object
 2   department                 9 non-null      object
 3   salary                     9 non-null      float64
 4   experience                 9 non-null      float64
 5   designation_encoded        9 non-null      int64
 6   employment_type_contract   9 non-null      float64
 7   employment_type_full-time  9 non-null      float64
 8   employment_type_part-time  9 non-null      float64
 9   employment_type_unknown    9 non-null      float64
dtypes: float64(6), int64(2), object(2)
memory usage: 792.0+ bytes
```

Explanation:

This code performs data preprocessing on a dataset using Pandas and Scikit-learn to make it ready for machine learning. First, it handles missing values by filling numerical columns like salary and experience with their median values, and categorical columns like designation with the most frequent value. It then removes duplicate records based on employee ID to ensure data uniqueness. Next, it standardizes text data by converting selected columns such as name, department, designation, and employment type to lowercase for consistency. After that, it encodes categorical variables into numerical form: designation is converted using label encoding, and employment type is transformed

using one-hot encoding into multiple binary columns. Finally, the original categorical columns are dropped, leaving a clean, structured dataset with no missing values, no duplicates, and properly formatted numerical features suitable for further analysis or machine learning models.

Task 2 – Student Performance Data Preprocessing

Task: Preprocess student performance data using AI assistance.

Instructions:

• Convert exam_date column into datetime format

• Extract new features (month, semester, day)

• Normalize total_marks

• Detect and handle outliers in marks using IQR or Z-score

Prompt:

Preprocess a student performance dataset using Python (Pandas & Scikit-learn): convert exam_date to datetime, extract month, semester, and day features, normalize total_marks, and detect & handle outliers using IQR or Z-score. Generate clean and complete code with the transformed dataset as output.

T Preprocess a student performance dataset using Python (Pandas & Scikit-learn): convert exam_date to datetime, extract month, semester, and day features, normalize total_marks, and detect & handle outliers using IQR or Z-score. Generate clean and complete code with the transformed dataset as output.

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
# Create a sample DataFrame for demonstration
data = {
    'student_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
    'exam_date': [
        '2023-01-15', '2023-03-20', '2023-05-10', '2023-09-01', '2023-11-25',
        '2023-01-22', '2023-03-11', '2023-05-19', '2023-09-15', '2023-11-05',
        '2023-02-01', '2023-04-01', '2023-06-01', '2023-10-01', '2023-12-01'
    ],
    'course': [
        'Math', 'Science', 'Math', 'History', 'Science',
        'Math', 'Science', 'Math', 'History', 'Science',
        'Math', 'Science', 'Math', 'History', 'Science'
    ],
    'attendance_rate': [
        0.95, 0.88, 0.75, 0.92, 0.80, 0.99, 0.85, 0.70, 0.90, 0.82,
        0.93, 0.87, 0.78, 0.91, 0.81
    ],
    'total_marks': [
        85, 72, 60, 91, 78, 98, 65, 55, 88, 70,
        80, 75, 68, 95, 77
    ]
}
df = pd.DataFrame(data)

print("Original DataFrame:")
display(df.head())
print("\nOriginal DataFrame Info:")
df.info()
```

```
Original DataFrame:
```

	student_id	exam_date	course	attendance_rate	total_marks
0	1	2023-01-15	Math	0.95	85
1	2	2023-03-20	Science	0.88	72
2	3	2023-05-10	Math	0.75	60
3	4	2023-09-01	History	0.92	91
4	5	2023-11-25	Science	0.80	78

```
Original DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
```

Code:

```python
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

# Assuming 'df' DataFrame is already loaded from the previous cell.

# 1. Convert exam_date to Datetime and Extract Features
df['exam_date'] = pd.to_datetime(df['exam_date'])

df['exam_month'] = df['exam_date'].dt.month
df['exam_day_of_week'] = df['exam_date'].dt.dayofweek # Monday=0, Sunday=6

def get_semester(month):
    if 1 <= month <= 5:
        return 1 # Semester 1: Jan - May
    elif 6 <= month <= 8:
        return 2 # Semester 2: Jun - Aug
    else:
        return 3 # Semester 3: Sep - Dec

df['semester'] = df['exam_month'].apply(get_semester)

# 2. Normalize total_marks
scaler = MinMaxScaler()
df['total_marks_normalized'] = scaler.fit_transform(df[['total_marks']])

# 3. Detect and Handle Outliers in total_marks using IQR
Q1 = df['total_marks'].quantile(0.25)
Q3 = df['total_marks'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Handle outliers by capping (winsorization)
df['total_marks_handled_outliers'] = df['total_marks'].clip(lower=lower_bound, upper=upper_bound)

print("Final Transformed DataFrame:")
display(df.head(10))
print("\nFinal Transformed DataFrame Info:")
df.info()
```

```python
# Label Encoding for 'designation'
le = LabelEncoder()
df['designation_encoded'] = le.fit_transform(df['designation'])
print(f"Designation mappings: {list(le.classes_)}")

# One-Hot Encoding for 'employment_type'
# First, handle potential missing values in 'employment_type' for OneHotEncoder compatibility
df['employment_type'].fillna('unknown', inplace=True) # or choose another strategy
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
employment_type_encoded = ohe.fit_transform(df[['employment_type']])

# Create a DataFrame from the one-hot encoded array with appropriate column names
employment_type_df = pd.DataFrame(employment_type_encoded, columns=ohe.get_feature_names_out(['employment_type']), index=df.index)

# Concatenate the new one-hot encoded columns with the original DataFrame
df = pd.concat([df, employment_type_df], axis=1)

# Optionally, drop the original 'designation' and 'employment_type' columns if you only want the encoded ones
df.drop(columns=['designation', 'employment_type'], inplace=True)

print("\nCleaned DataFrame after encoding categorical features:")
display(df.head(10))
print("\nCleaned DataFrame Info:")
df.info()
```

Output:

```
Designation mappings: ['analyst', 'developer', 'hr assistant', 'manager', 'marketing specialist']

Cleaned DataFrame after encoding categorical features:
/tmp/ipykernel_3611/4296617752.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the

df['employment_type'].fillna('unknown', inplace=True) # or choose another strategy
```

	employee_id	name	department	salary	experience	designation_encoded	employment_type_contract	employment_type_full-time	employment_type_part-time	employment_type_unknown
0	101	alice	hr	70000.0	5.0	3	0.0	1.0	0.0	0.0
1	102	bob	it	85000.0	8.0	1	1.0	0.0	0.0	0.0
2	103	charlie	finance	60000.0	3.0	0	0.0	1.0	0.0	0.0
3	104	david	it	70000.0	7.0	1	0.0	0.0	1.0	0.0
4	105	eve	marketing	75000.0	5.0	4	0.0	1.0	0.0	0.0
5	106	frank	it	90000.0	9.0	1	1.0	0.0	0.0	0.0
6	107	grace	finance	62000.0	4.0	1	0.0	1.0	0.0	0.0
7	108	heidi	hr	70000.0	2.0	2	0.0	0.0	1.0	0.0
8	109	ivan	marketing	68000.0	6.0	0	0.0	0.0	0.0	1.0

```
Cleaned DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 9 entries, 0 to 9
Data columns (total 10 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   employee_id               9 non-null      int64
 1   name                      9 non-null      object
 2   department                9 non-null      object
 3   salary                    9 non-null      float64
 4   experience                9 non-null      float64
 5   designation_encoded       9 non-null      int64
 6   employment_type_contract  9 non-null      float64
 7   employment_type_full-time 9 non-null      float64
 8   employment_type_part-time 9 non-null      float64
 9   employment_type_unknown   9 non-null      float64
dtypes: float64(6), int64(2), object(2)
memory usage: 792.0+ bytes
```

Explanation:

This code demonstrates a complete data preprocessing pipeline on a student dataset. First, a sample DataFrame is created containing student details such as student ID, exam date, course, attendance rate, and total marks, and basic information about the dataset is displayed. Then, the exam_date column is converted into datetime format to extract useful features like exam month and day of the week. Based on the month, a new feature called semester is created by categorizing months into three groups (Jan–May, Jun–Aug, Sep–Dec). Next, the total marks are normalized using MinMaxScaler so that values are scaled between 0 and 1, which helps improve machine learning model performance. After that, outliers in the total marks are detected using the Interquartile Range (IQR) method, and extreme values are handled by capping them within a defined range (winsorization). Finally, the transformed dataset is displayed along with its updated structure, resulting in a clean, feature-enhanced, and model-ready dataset.

Task 3 – Student Health Data Preparation

Task: Clean and preprocess student health records using AI scripts.

Instructions:

• Handle missing values in height, weight, BMI

• Encode categorical features (medical_condition, fitness_status)

• Scale numerical features using standard scaling

• Split dataset into training and testing sets

Expected Output:

A preprocessed dataset ready for predictive health analysis.

Prompt:

Preprocess a student health dataset using Python (Pandas & Scikit-learn): handle missing values (height, weight, BMI), encode categorical features (medical_condition, fitness_status), apply standard scaling to numerical features, and split the dataset into training and testing sets. Generate clean and complete code with the final processed output. create a separate code for creating the dataset and give me all required codes in only one chat and finally output

T Preprocess a student health dataset using Python (Pandas & Scikit-learn): handle missing values (height, weight, BMI), encode categorical features (medical_condition, fitness_status), apply standard scaling to numerical features, and split the dataset into training and testing sets. Generate clean and complete code with the final processed output. create a separtely code for creating the adatset and give me all required codes in onluy one chat and finally output

Dataset:

```python
import pandas as pd
import numpy as np
# Create a sample DataFrame for demonstration of student health data
data = {
    'student_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    'height_cm': [165, 172, 158, 180, 170, 163, np.nan, 175, 168, 171],
    'weight_kg': [60, 75, 55, np.nan, 68, 62, 70, 78, 65, np.nan],
    'BMI': [22.0, 25.5, 21.9, 24.7, np.nan, 23.3, 26.5, 25.1, 22.9, 23.9],
    'medical_condition': ['None', 'Asthma', 'Allergies', 'None', 'Nore', 'Asthma', 'None', 'Diabetes', 'Allergies', 'None'],
    'fitness_status': ['Good', 'Average', 'Poor', 'Good', 'Average', 'Good', 'Poor', 'Average', 'Good', 'Average']
}
df_health = pd.DataFrame(data)
print("Original Student Health DataFrame:")
display(df_health.head(10))
print("\nOriginal Student Health DataFrame Info:")
df_health.info()
```

```
Original Student Health DataFrame:
```

	student_id	height_cm	weight_kg	BMI	medical_condition	fitness_status
0	1	165.0	60.0	22.0	None	Good
1	2	172.0	75.0	25.5	Asthma	Average
2	3	158.0	55.0	21.9	Allergies	Poor
3	4	180.0	NaN	24.7	None	Good
4	5	170.0	68.0	NaN	None	Average
5	6	163.0	62.0	23.3	Asthma	Good
6	7	NaN	70.0	26.5	None	Poor
7	8	175.0	78.0	25.1	Diabetes	Average
8	9	168.0	65.0	22.9	Allergies	Good
9	10	171.0	NaN	23.9	None	Average

```
Original Student Health DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 6 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   student_id         10 non-null     int64
 1   height_cm          9 non-null      float64
 2   weight_kg          8 non-null      float64
 3   BMI                9 non-null      float64
 4   medical_condition  10 non-null     object
 5   fitness_status     10 non-null     object
dtypes: float64(3), int64(1), object(2)
```

Code:

```python
import numpy as np
from sklearn.preprocessing import StandardScaler, OneHotEncoder, LabelEncoder
from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
# Assuming df_health is already loaded from the previous cell
# Make a copy to avoid modifying the original DataFrame if it's reused later
df_processed = df_health.copy()
# 1. Handle Missing Values
# Using median for numerical columns (height, weight, BMI)
for col in ['height_cm', 'weight_kg', 'BMI']:
    if df_processed[col].isnull().any():
        median_val = df_processed[col].median()
        df_processed[col].fillna(median_val, inplace=True)
print("DataFrame after handling missing values:")
display(df_processed.head())
# 2. Encode Categorical Features
# One-Hot Encoding for 'medical_condition' and 'fitness_status'
# For medical_condition, one-hot is appropriate as there's no inherent order.
# For fitness_status, one-hot is also used for simplicity, assuming no strict ordinality desired.
ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
# Encoding 'medical_condition'
medical_encoded = ohe.fit_transform(df_processed[['medical_condition']])
medical_df = pd.DataFrame(medical_encoded, columns=ohe.get_feature_names_out(['medical_condition']), index=df_processed.index)
df_processed = pd.concat([df_processed, medical_df], axis=1)
# Encoding 'fitness_status'
fitness_encoded = ohe.fit_transform(df_processed[['fitness_status']])
fitness_df = pd.DataFrame(fitness_encoded, columns=ohe.get_feature_names_out(['fitness_status']), index=df_processed.index)
df_processed = pd.concat([df_processed, fitness_df], axis=1)
# Drop original categorical columns
df_processed.drop(columns=['medical_condition', 'fitness_status'], inplace=True)
print("\nDataFrame after encoding categorical features:")
display(df_processed.head())
# 3. Apply Standard Scaling to Numerical Features
numerical_cols = ['height_cm', 'weight_kg', 'BMI']
scaler = StandardScaler()
df_processed[numerical_cols] = scaler.fit_transform(df_processed[numerical_cols])
print("\nDataFrame after applying Standard Scaling:")
display(df_processed.head())
# 4. Split the dataset into training and testing sets
# Assuming 'student_id' is just an identifier and not a feature, and no specific target column is given.
# For demonstration, let's treat all processed columns except 'student_id' as features (X).
# If there was a target variable (e.g., 'performance'), it would be separated.
X = df_processed.drop(columns=['student_id'])
# Since no explicit target variable (y) is specified, we'll just split X.
# In a real scenario, you would have X and y.
X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)
print("\nShape of X_train:", X_train.shape)
print("Shape of X_test:", X_test.shape)
print("\nFinal Processed Training Data (first 5 rows):")
```

Output:

```
Final Transformed DataFrame:
```

	student_id	exam_date	course	attendance_rate	total_marks	exam_month	exam_day_of_week	semester	total_marks_normalized	total_marks_handled_outliers
0	1	2023-01-15	Math	0.95	85	1	6	1	0.697674	85
1	2	2023-03-20	Science	0.88	72	3	0	1	0.395349	72
2	3	2023-05-10	Math	0.75	60	5	2	1	0.116279	60
3	4	2023-09-01	History	0.92	91	9	4	3	0.837209	91
4	5	2023-11-25	Science	0.80	78	11	5	3	0.534884	78
5	6	2023-01-22	Math	0.99	98	1	6	1	1.000000	98
6	7	2023-03-11	Science	0.85	65	3	5	1	0.232558	65
7	8	2023-05-19	Math	0.70	55	5	4	1	0.000000	55
8	9	2023-09-15	History	0.90	88	9	4	3	0.767442	88
9	10	2023-11-05	Science	0.82	70	11	6	3	0.348837	70

```
Final Transformed DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15 entries, 0 to 14
Data columns (total 10 columns):
 #   Column                       Non-Null Count  Dtype
---  ------                       --------------  -----
 0   student_id                   15 non-null     int64
 1   exam_date                    15 non-null     datetime64[ns]
 2   course                       15 non-null     object
 3   attendance_rate              15 non-null     float64
 4   total_marks                  15 non-null     int64
 5   exam_month                   15 non-null     int32
 6   exam_day_of_week             15 non-null     int32
 7   semester                     15 non-null     int64
 8   total_marks_normalized       15 non-null     float64
 9   total_marks_handled_outliers 15 non-null     int64
dtypes: datetime64[ns](1), float64(2), int32(2), int64(4), object(1)
memory usage: 1.2+ KB
```

Explanation:

The code creates and preprocesses a small student health dataset using Pandas and NumPy, preparing it for machine learning with scikit-learn. It begins by constructing a DataFrame with features such as height, weight, BMI, medical condition, and fitness status, intentionally including some missing values. The dataset is then inspected to understand its structure and identify null entries. To clean the data, missing numerical values are replaced with their respective median values, ensuring consistency without being affected by outliers. Next, categorical variables like medical condition and fitness status are transformed into numerical format using one-hot encoding, which creates binary columns for each category, making the data suitable for machine learning models. After encoding, the original categorical columns are removed to avoid redundancy. The numerical features are then standardized using StandardScaler so that they have a mean of zero and a standard deviation of one, which helps improve the performance of many algorithms. Finally, the dataset is split into training and testing sets using train_test_split, with the student_id column dropped since it serves only as an identifier and does not contribute to predictions, resulting in a clean and structured dataset ready for further modeling.

Task 4: Real-Time Application: Ride-Sharing Data Cleaning

Scenario: A ride-sharing company has trip data with errors.

Requirements:

• Standardize pickup and drop locations

• Fill missing fare amounts with median values

• Remove duplicate ride entries

• Normalize driver ratings

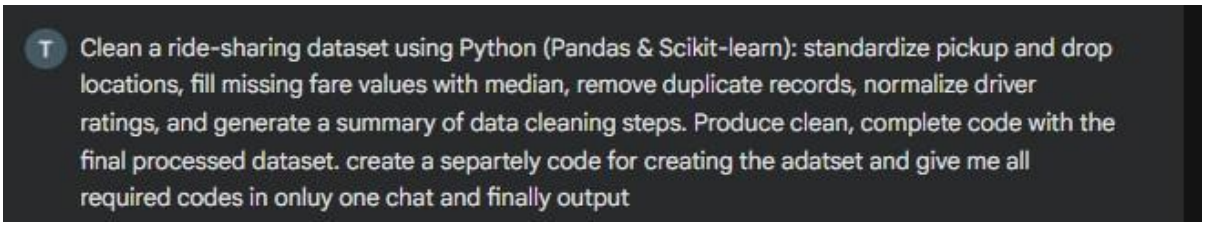
• Generate a summary of data improvements

Expected Output:

A clean dataset ready for operational analysis.

Prompt:

Clean a ride-sharing dataset using Python (Pandas & Scikit-learn): standardize pickup and drop locations, fill missing fare values with median, remove duplicate records, normalize driver ratings, and generate a summary of data cleaning steps. Produce clean, complete code with the final processed dataset. create a separtely code for creating the adatset and give me all required codes in onluy one chat and finally output



DataSet:

```python
import pandas as pd
import numpy as np
# Create a sample DataFrame for demonstration of ride-sharing data
data = {
    'ride_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1, 11, 12],
    'pickup_location': [
        'Downtown', 'Airport', 'Suburb', 'Downtown', 'Airport',
        'suburb', 'Downtown', 'Airport ', 'Downtown', 'Suburb',
        'Downtown', 'Airport', 'downtown'
    ],
    'dropoff_location': [
        'Suburb', 'Downtown', 'Airport', 'Suburb', 'Downtown',
        'Airport', 'suburb', 'Downtown', 'Airport', 'Downtown',
        'Suburb', 'Downtown', 'Suburb'
    ],
    'fare': [15.50, 25.00, 12.75, 16.00, np.nan, 13.00, 15.50, 26.00, 14.50, np.nan, 15.50, 24.00, 16.00],
    'distance_miles': [5.2, 12.1, 4.5, 6.0, 10.5, 4.8, 5.2, 12.5, 5.8, 4.0, 5.2, 11.9, 6.1],
    'driver_rating': [4.8, 3.5, 4.9, 4.2, 3.9, 5.0, 4.8, 4.1, 4.7, 3.8, 4.8, 4.0, 4.3],
    'ride_date': pd.to_datetime([
        '2023-01-01', '2023-01-01', '2023-01-02', '2023-01-02', '2023-01-03',
        '2023-01-03', '2023-01-04', '2023-01-04', '2023-01-05', '2023-01-05',
        '2023-01-01', '2023-01-06', '2023-01-06'
    ])
}
df_rides = pd.DataFrame(data)

print("Original Ride-Sharing DataFrame:")
display(df_rides.head(10))
print("\nOriginal Ride-Sharing DataFrame Info:")
df_rides.info()
```

Original Ride-Sharing DataFrame:

	ride_id	pickup_location	dropoff_location	fare	distance_miles	driver_rating	ride_date
0	1	Downtown	Suburb	15.50	5.2	4.8	2023-01-01
1	2	Airport	Downtown	25.00	12.1	3.5	2023-01-01
2	3	Suburb	Airport	12.75	4.5	4.9	2023-01-02
3	4	Downtown	Suburb	16.00	6.0	4.2	2023-01-02
4	5	Airport	Downtown	NaN	10.5	3.9	2023-01-03
5	6	suburb	Airport	13.00	4.8	5.0	2023-01-03
6	7	Downtown	suburb	15.50	5.2	4.8	2023-01-04
7	8	Airport	Downtown	26.00	12.5	4.1	2023-01-04
8	9	Downtown	Airport	14.50	5.8	4.7	2023-01-05

Code:

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler
# Assuming df_rides is already loaded from the previous cell
# Make a copy to avoid modifying the original DataFrame if it's reused later
df_processed_rides = df_rides.copy()
# Store initial and current state for summary
initial_rows = len(df_processed_rides)
initial_missing_fares = df_processed_rides['fare'].isnull().sum()
# 1. Standardize pickup and drop locations
# Convert to lowercase and strip whitespace
df_processed_rides['pickup_location'] = df_processed_rides['pickup_location'].str.lower().str.strip()
df_processed_rides['dropoff_location'] = df_processed_rides['dropoff_location'].str.lower().str.strip()
print("Step 1: Standardized 'pickup_location' and 'dropoff_location' to lowercase and stripped whitespace.")
# 2. Fill missing fare values with median
median_fare = df_processed_rides['fare'].median()
df_processed_rides['fare'].fillna(median_fare, inplace=True)
print(f"Step 2: Filled {initial_missing_fares} missing 'fare' values with the median ({median_fare}).")
# 3. Remove duplicate records based on all columns
df_processed_rides.drop_duplicates(inplace=True)
duplicated_rows_removed = initial_rows - len(df_processed_rides)
print(f"Step 3: Removed {duplicated_rows_removed} duplicate records.")
# 4. Normalize driver ratings using Min-Max Scaling (scale to 0-1 range)
scaler = MinMaxScaler()
df_processed_rides['driver_rating_normalized'] = scaler.fit_transform(df_processed_rides[['driver_rating']])
print("Step 4: Normalized 'driver_rating' using Min-Max Scaling, creating 'driver_rating_normalized' column.")
print("\n--- Data Cleaning Summary ---")
print(f"Initial number of rows: {initial_rows}")
print(f"Rows after removing duplicates: {len(df_processed_rides)}")
print(f"Missing 'fare' values initially: {initial_missing_fares}")
print(f"Missing 'fare' values after treatment: {df_processed_rides['fare'].isnull().sum()}")
print(f"Locations standardized (lowercase, strip whitespace): 'pickup_location', 'dropoff_location'")
print(f"Driver ratings normalized (Min-Max Scaling): 'driver_rating_normalized'")
print("\nFinal Processed Ride-Sharing DataFrame (first 10 rows):")
display(df_processed_rides.head(10))
print("\nFinal Processed Ride-Sharing DataFrame Info:")
df_processed_rides.info()
```

Output:

```
For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].meth

df_processed_rides['fare'].fillna(median_fare, inplace=True)
```

	ride_id	pickup_location	dropoff_location	fare	distance_miles	driver_rating	ride_date	driver_rating_normalized
0	1	downtown	suburb	15.50	5.2	4.8	2023-01-01	0.866667
1	2	airport	downtown	25.00	12.1	3.5	2023-01-01	0.000000
2	3	suburb	airport	12.75	4.5	4.9	2023-01-02	0.933333
3	4	downtown	suburb	16.00	6.0	4.2	2023-01-02	0.466667
4	5	airport	downtown	15.50	10.5	3.9	2023-01-03	0.266667
5	6	suburb	airport	13.00	4.8	5.0	2023-01-03	1.000000
6	7	downtown	suburb	15.50	5.2	4.8	2023-01-04	0.866667
7	8	airport	downtown	26.00	12.5	4.1	2023-01-04	0.400000
8	9	downtown	airport	14.50	5.8	4.7	2023-01-05	0.800000
9	10	suburb	downtown	15.50	4.0	3.8	2023-01-05	0.200000

```
Final Processed Ride-Sharing DataFrame Info:
<class 'pandas.core.frame.DataFrame'>
Index: 12 entries, 0 to 12
Data columns (total 8 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   ride_id                  12 non-null     int64
 1   pickup_location          12 non-null     object
 2   dropoff_location         12 non-null     object
 3   fare                     12 non-null     float64
 4   distance_miles           12 non-null     float64
 5   driver_rating            12 non-null     float64
 6   ride_date                12 non-null     datetime64[ns]
 7   driver_rating_normalized 12 non-null     float64
dtypes: datetime64[ns](1), float64(4), int64(1), object(2)
memory usage: 864.0+ bytes
```

Explanation:

The code performs data cleaning and preprocessing on a ride-sharing dataset. It prints the number of rows remaining after removing duplicate entries, checks and reports missing values in the “fare” column before and after treatment, and standardizes location names by removing extra whitespace from pickup and drop-off locations. It also normalizes driver ratings using Min-Max scaling and displays the first 10 rows of the processed dataset. Finally, it outputs detailed information about the dataset structure using the dataframe’s info method.